

GOOGLE 12-MONTH CAREER & INTERVIEW BLUEPRINT

For Software Engineering, AI/ML, GenAI, Cloud & Full-Stack Targets

Target roles: Google Ads AI/ML GenAI | Google Cloud AI/ML | Google Cloud Full Stack | YouTube Creator
Support | Applied AI/ML

12 months | 2-3 hours/day alongside a current full-time job

Core objective

Become a software engineer who can build production systems, understands ML deeply enough to own the lifecycle, can build GenAI/agent systems, can design scalable services, and can pass Google-style DSA and system-design interviews.

1. FIRST: WHICH GOOGLE ROLE SHOULD BE YOUR PRIMARY TARGET?

This should not be treated as a generic “learn AI + do LeetCode” plan. The right strategy is to build one profile capable of applying to all five roles, rather than preparing separately for five jobs.

Priority	Role	Target difficulty	Why
1	Software Engineer III, Full Stack - Google Cloud	Best initial target	Strong software engineering + full-stack; easier to establish objectively in portfolio
2	Applied AI/ML Engineer	Very strong target	AI/ML education + automation background can become a strong story
3	Senior SWE, AI/ML - Google Cloud	Stretch target	Requires strong ML infrastructure + software architecture
4	Senior SWE, AI/ML GenAI - Google Ads	Stretch target	Strong GenAI + ML infra + software engineering required
5	Senior SWE, YouTube Creator Support	Stretch target	Strong LLM production systems + senior SWE expectations

The common denominator is: software engineering, Python/C++/strong programming, production ownership, ML infrastructure, system design, and (for several targets) GenAI/LLM systems. The Full Stack path additionally requires frontend JavaScript/TypeScript.

The important point is that you already have a useful starting base: software/automation experience, Python/C, ML/AI education, and automotive engineering experience. The gap is turning that into a Google-shaped engineering profile with demonstrable production AI, strong DSA, system design, cloud deployment, and a compelling public portfolio.

2. YOUR TARGET PROFILE AFTER 12 MONTHS

By the end of these 12 months, your profile should look approximately like this:

Core SWE

- Python - advanced
- Data structures & algorithms - interview ready
- OOP/design patterns
- Multithreading/concurrency
- Linux
- Git
- Testing
- Debugging
- APIs
- Databases
- Networking fundamentals

Backend

- FastAPI
- PostgreSQL
- Redis
- asynchronous programming
- REST APIs
- authentication/authorization
- background workers
- queues
- caching
- rate limiting

Frontend

- TypeScript
- React
- HTML/CSS
- accessibility
- state management
- API integration

ML

- classical ML
- deep learning
- PyTorch
- model evaluation
- experimentation
- feature engineering
- data pipelines
- model deployment
- model versioning
- model monitoring
- drift detection
- inference optimization

GenAI

- Transformers
- embeddings
- vector databases
- RAG
- hybrid retrieval
- reranking
- tool calling
- agents
- structured outputs
- evaluation
- guardrails
- hallucination mitigation
- prompt/version management
- human-in-the-loop
- observability

Cloud

- GCP
- Cloud Run
- GKE
- Pub/Sub
- Cloud Storage
- Cloud SQL
- BigQuery
- Artifact Registry
- IAM
- Secret Manager
- Vertex AI concepts
- monitoring/logging

Production engineering

- Docker
- Kubernetes
- CI/CD

- GitHub Actions
- Terraform basics
- logging
- metrics
- tracing
- alerting
- canary deployments
- rollback
- load testing

Production ML is much more than the model: data validation, feature processing, deployment, monitoring, testing, resource management and serving are major parts of the system.

3. THE MOST IMPORTANT DECISION: DO NOT BUILD 15 SMALL PROJECTS

Build three major projects and perhaps two smaller supporting projects. Your portfolio should demonstrate depth.

PROJECT 1 - ENGINEERING CHANGE INTELLIGENCE PLATFORM

This should become your signature project. It is closely related to the automotive intelligent software lifecycle work you have already been developing, but you should make a clean public version without exposing proprietary company information.

Think: “AI Engineering Change & Test Intelligence Platform.”

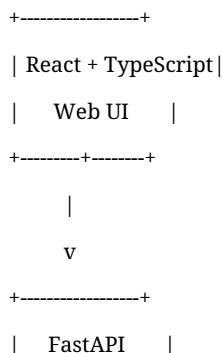
A developer uploads:

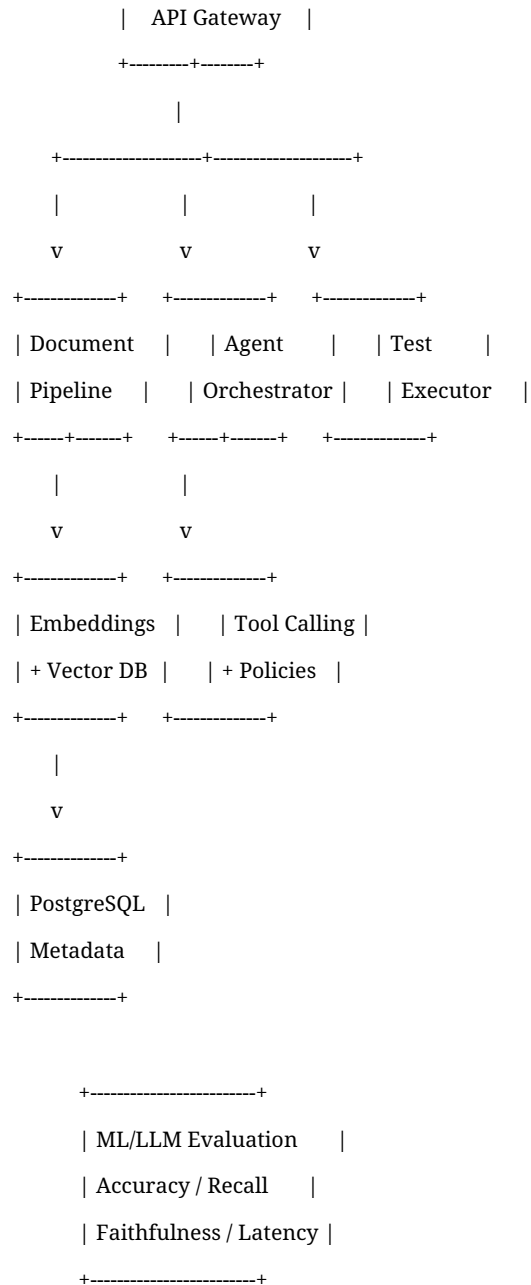
- software change request
- requirements
- architecture documents
- source code
- previous test cases
- specifications

The system:

- understands the change
- retrieves relevant historical knowledge
- identifies impacted components
- generates candidate test cases
- generates executable test inputs
- runs tests
- analyzes failures
- explains failures
- updates its knowledge base
- provides traceability from requirement -> code -> test -> result

Architecture





Technology stack

Frontend

- React
- TypeScript
- Vite
- Tailwind or plain CSS

Backend

- Python
- FastAPI
- Pydantic
- PostgreSQL
- Redis

AI

- PyTorch
- Hugging Face
- sentence-transformers
- vector database
- LLM API
- RAG
- reranking

Infrastructure

- Docker
- Kubernetes
- GitHub Actions
- GCP
- Cloud Storage
- Cloud SQL
- GKE/Cloud Run

Observability

- OpenTelemetry
- Prometheus
- Grafana
- structured logging

Phase 1 - ingestion

Build POST /documents, GET /documents, GET /documents/{id}, DELETE /documents/{id}. Accept PDF, DOCX, TXT, Markdown and CSV. Create documents, document_chunks and document_metadata tables. Implement document -> text extraction -> cleaning -> chunking -> embedding -> vector storage. Add release/component/document type/version metadata.

Phase 2 - RAG

Implement baseline RAG, then improve it: Version 1 dense retrieval; Version 2 hybrid BM25 + vector search; Version 3 reranking; Version 4 metadata filtering; Version 5 query decomposition. Measure retrieval quality with Recall@K, MRR and nDCG.

Phase 3 - agent

Create tools: search_requirements(), search_code(), search_previous_tests(), analyze_change(), generate_test_case(), run_test(), inspect_test_result(), compare_releases(), create_report(). Use a planner -> policy layer -> tool -> tool result -> LLM architecture. Add maximum iterations, timeout, tool allowlist, input validation and output validation.

Phase 4 - human-in-the-loop

Add approval before risky operations. Example: if confidence > 90%, auto-execute; otherwise route to human approval. For destructive actions require approval. This makes the project more realistic for production agentic systems.

Phase 5 - evaluation

Measure retrieval Recall@K, MRR, nDCG; RAG faithfulness, answer relevance, citation correctness; agent task success rate, tool selection accuracy, average steps, failure rate; system latency, throughput, cost/request and memory/CPU.

Phase 6 - observability

Every request should record request_id, user_id, model_version, prompt_version, retrieval_version, latency, tokens, tools_used, errors and final_result. Add traces for request, retrieval, embedding, vector search, reranking, LLM call, tool call and final response.

PROJECT 2 - PRODUCTION ML PLATFORM

This project demonstrates that you are an ML engineer, not merely an LLM application developer.

Name it something like “ModelForge - Production ML Training & Serving Platform.”

Dataset

|

v

Validation

|

v

Feature engineering

|

v

Training

|

v

Experiment tracking

|

v

Model evaluation

|

v

Model registry

|

v

Deployment

|

v

Canary

|

v

Monitoring

|

v

Drift

|

v

Retraining

Data pipeline

Implement dataset ingestion -> schema validation -> missing-value checks -> distribution checks -> feature generation -> dataset version.

Training

Support Logistic Regression, XGBoost/LightGBM and a PyTorch neural network. Track model version, data version, code commit, hyperparameters, metrics and training time. Use MLflow.

Model registry

Create versions such as v1, v2, v3 with accuracy, precision, recall, F1, latency, training dataset, git SHA and status. Statuses: TRAINED, VALIDATED, STAGING, PRODUCTION, ARCHIVED.

Serving

Build POST /predict, GET /health, GET /model and GET /metrics. Containerize it. Run locally with Docker, then Kubernetes, then GCP.

Canary deployment

Route 95% to current v1 and 5% to candidate v2, then progressively move 5% -> 20% -> 50% -> 100% while monitoring latency, error rate, prediction distribution and business metrics, with rollback capability.

Drift detection

Implement PSI, KS test, feature distribution drift and prediction drift. Example: detect engine_temperature PSI = 0.31 against threshold 0.20 and trigger retraining investigation.

PROJECT 3 - FULL-STACK AI DEVELOPER PLATFORM

This project is designed to close the Google Cloud Full Stack role gap. Call it "AI Engineering Command Center." It becomes the frontend/control plane for Projects 1 and 2.

Frontend

- React + TypeScript
- Vite
- component architecture
- state management
- forms
- API integration
- authentication
- accessibility
- responsive design
- Pages: Dashboard, Projects, Datasets, Models, Experiments, Deployments, Agents, Evaluations, Logs, Alerts, Users

Backend

- FastAPI
- auth
- projects
- datasets
- models
- experiments
- deployments
- agents
- evaluations
- monitoring

Platform

- PostgreSQL
- Redis
- Celery/Redis or Pub/Sub
- JWT/OAuth concepts
- OpenAPI
- Docker
- Kubernetes
- GCP

Conceptual dashboard:

+-----+

| AI Engineering Command Center |

+-----+-----+		
Projects	Model health	
Models	Agent success rate	
Agents	Request latency	
Runs	Evaluation results	
Deploy	Drift alerts	
Logs	Recent failures	
+-----+-----+		

4. YOUR 12-MONTH ROADMAP

The schedule should survive alongside your current job. Recommended: 2 hours/day Monday-Friday, 3 hours Saturday, 3 hours Sunday. Total: about 16 hours/week, or 800+ focused hours/year.

Daily structure

Day	Focus
Monday	60 min DSA; 60 min project
Tuesday	60 min DSA; 60 min ML/AI theory
Wednesday	30 min DSA; 90 min project
Thursday	60 min system design; 60 min project
Friday	60 min DSA; 60 min project
Saturday	3 hours project - no theory
Sunday	1 hour review; 1 hour reading/documentation; 1 hour interview practice

MONTH 1 - Software engineering foundation

Goal: Become extremely comfortable writing production-quality Python.

Learn Python internals, OOP, dataclasses, typing, generators, decorators, context managers, exceptions, iterators, async/await, multiprocessing, threading, unit testing, pytest, logging.

Also learn Linux, Git, GitHub, virtual environments and packaging.

Build python-engineering-playground with an LRU cache, rate limiter, retry mechanism, task queue, thread pool, connection pool, logger and configuration manager. Do not just copy implementations; write them yourself.

MONTH 2 - DSA foundations

Focus on arrays, strings, hashing, linked lists, stacks, queues and recursion. Learn two pointers, sliding window, prefix sums, binary search, frequency maps, sets and parsing.

Target 50-60 quality problems, with a note for Problem, Pattern, Brute force, Optimized, Complexity, Mistake and Alternative. Goal is pattern recognition.

MONTH 3 - DSA + backend

Start Project 1. Learn HTTP, REST, JSON, status codes, REST semantics, authentication, API design, FastAPI, PostgreSQL, SQL, indexes and transactions.

Build Document Service with POST /documents, GET /documents, GET /documents/{id}, DELETE /documents/{id}. Write unit tests, integration tests and API tests.

End of Month 3: ~120 DSA problems and working backend service #1.

MONTH 4 - Machine learning engineering

Learn linear regression, logistic regression, trees, random forests, gradient boosting, clustering, PCA, metrics, cross validation, feature engineering, class imbalance and calibration.

Build dataset -> training -> validation -> evaluation pipelines with pandas, scikit-learn and PyTorch. Create experiments, compare models and write a technical report.

MONTH 5 - Deep learning + ML infrastructure

Learn tensors, autograd, optimization, backpropagation, CNNs, attention and transformers, then move directly into MLflow, model versioning, experiment tracking, Docker and inference API.

Start Project 2 with train.py, evaluate.py and serve.py, then Docker and docker-compose.

MONTH 6 - RAG mastery

Build Project 1 seriously. Learn embeddings, vector search, chunking, metadata, semantic search, BM25, hybrid retrieval, reranking and contextual compression.

Compare fixed vs semantic chunks; vector vs hybrid search; hybrid vs hybrid + reranker. Record measurements. GitHub should show graphs, tables and conclusions - not just "Built a RAG chatbot."

MONTH 7 - Agent engineering

Turn Project 1 into a real agent. Learn planning, tool calling, state, memory, workflow orchestration, retries, timeouts, idempotency, tool validation and agent evaluation.

Build Requirement Analyst Agent -> Impact Analysis Agent -> Test Generation Agent -> Execution Agent -> Failure Analysis Agent. Use an orchestrated workflow with clear responsibilities. Add human approval before risky actions.

MONTH 8 - Full-stack

Start Project 3. Learn TypeScript, React, component architecture, state management, forms, API integration, authentication, accessibility and responsive design.

Build the AI Engineering Command Center dashboard.

MONTH 9 - Cloud + Kubernetes

Deploy everything. Learn Docker, Kubernetes pods, deployments, services, config maps, secrets, ingress, autoscaling and health checks.

Learn GCP: Cloud Storage, Cloud SQL, Cloud Run, GKE, Pub/Sub, Artifact Registry, IAM, Secret Manager, Cloud Logging and Cloud Monitoring.

Create push -> GitHub Actions -> test -> build -> Docker -> registry -> deploy CI/CD.

MONTH 10 - Production hardening

Stop building demos; make the projects production-oriented. Add unit, integration, API, end-to-end and load tests. Add metrics, logs and traces.

Implement secrets management, RBAC, input validation, rate limiting, retries, circuit breakers, timeouts and graceful degradation. Add ML drift, model quality, latency and error-rate monitoring.

MONTH 11 - System design + interview mode

Stop adding major functionality; projects should be ~90% complete. Focus on fundamental systems: URL shortener, rate limiter, notification system, chat system, file storage, task scheduler.

Then AI systems: RAG system, LLM inference service, vector search platform, recommendation system, feature store, model serving platform, distributed training platform, AI agent platform.

For every design practice requirements -> scale -> API -> data model -> architecture -> bottlenecks -> caching -> partitioning -> consistency -> reliability -> monitoring -> cost.

MONTH 12 - Google interview simulation

Four DSA sessions/week, two system-design sessions/week, one behavioral session/week, one project deep-dive/week. Use timed practice.

You are no longer “learning”; you are preparing to perform. Be able to explain your flagship project for 5, 15 and 30 minutes without preparation.

5. YOUR DSA TARGET

Do not chase problem count. Build pattern recognition and revisit problems.

Period	Target
Months 1-3	150 problems
Months 4-6	100 problems
Months 7-9	100 problems
Months 10-12	100 problems
Total	~450 carefully selected problems

Problem families:

- Sliding window
- Two pointers
- Binary search
- Hashing
- Stack
- Monotonic stack
- Heap
- Intervals
- Trees
- BFS
- DFS
- Backtracking
- Graphs
- Topological sort
- Union-find
- Dynamic programming
- Greedy
- Trie
- Prefix sum
- Design/data-structure questions

6. DO NOT OVERSPREAD YOUR LANGUAGE STACK

You should not spend the year trying to become equally strong in Python, C++, Java, Go and TypeScript.

Use	Primary language
Interview	Python
AI/ML	Python / PyTorch
Backend	Python / FastAPI
Frontend	TypeScript / React
Existing domain expertise	C

You can learn enough C++ later if needed. The linked AI/ML roles accept Python or C++ as programming languages, so Python is a rational primary language for this roadmap.

7. YOUR RESUME STRATEGY NEEDS TO CHANGE

Do not write your resume as: “Automation Expert with 5 years of experience working in automotive automation, Python, AI/ML and testing.” That is too generic.

Your resume should communicate: “Software engineer building AI/ML-enabled engineering systems with production automation, testing, data, and cloud experience.”

Your experience needs to show five dimensions:

Software

+

AI/ML

+

Production

+

Architecture

+

Impact

Convert your current job into Google language, but only use claims genuinely supported by your work.

- Instead of “Developed automation scripts for ECU testing,” aim for evidence like: “Designed and implemented automated software validation workflows for ECU releases, reducing manual validation effort and improving regression-test coverage.”
- Instead of “Worked with Python,” use evidence like: “Built Python-based automation services integrating software-change workflows, test execution and result analysis.”
- Instead of “Created ML model,” use evidence like: “Designed an ML pipeline for X, including data preprocessing, model evaluation and automated inference.”

A strong flagship Project 1 bullet should eventually resemble:

Built a production-oriented AI engineering platform using Python, FastAPI, PostgreSQL, vector search and LLMs to analyze software changes, retrieve engineering knowledge, generate validation scenarios and execute tool-driven workflows with human approval.

Then quantify actual measured improvements, for example retrieval Recall@K, irrelevant-context reduction, latency or agent success rate. Do not invent numbers.

8. RESUME STRUCTURE

Header

NAME

Software Engineer | AI/ML | GenAI | Distributed Systems

Bengaluru, India

GitHub | LinkedIn | Portfolio

Summary: three lines maximum. Example positioning: “Software engineer with 5+ years of experience building automation and software engineering systems, with advanced training in AI/ML. Focused on production ML, GenAI, agentic systems, backend engineering and scalable cloud services.”

Skills section:

Languages

Python, C, TypeScript, Java

AI/ML

PyTorch, scikit-learn, Transformers, RAG, LLMs, MLflow

Backend

FastAPI, PostgreSQL, Redis, REST

Cloud/Infrastructure

GCP, Docker, Kubernetes, GitHub Actions

Frontend

React, TypeScript, HTML, CSS

Engineering

Linux, Git, CI/CD, testing, observability

9. GITHUB STRATEGY

Pin only:

- AI Engineering Intelligence Platform
- ModelForge
- AI Engineering Command Center
- DSA repository
- One research/ML project

Every major repository should contain:

README

architecture.png

docs/

src/

tests/

deployment/

benchmark/

experiments/

README sections: Problem, Motivation, Architecture, Technical decisions, Dataset, Approach, Experiments, Results, Deployment, Monitoring, Failure modes and Future work.

10. WRITE ENGINEERING DOCUMENTS

For every flagship project create:

requirements.md

architecture.md

api.md

database.md

security.md

evaluation.md

deployment.md

observability.md

You want reviewers to see that you do not merely make demos. You engineer systems.

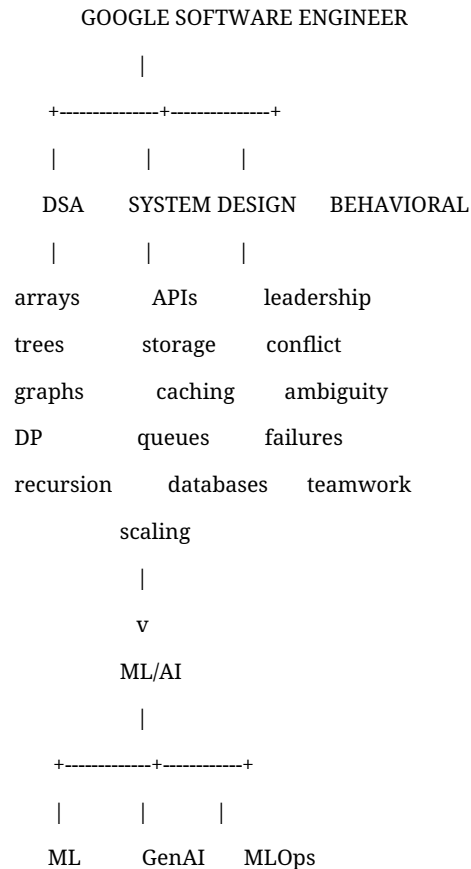
11. SYSTEM DESIGN PREPARATION SHOULD BE TIED TO YOUR PROJECTS

Do not study system design independently for months. Take Project 1 and repeatedly scale the thought experiment.

- What happens when 10 users use it?
- What happens at 1,000?
- What happens at 100,000?
- What happens at 10 million documents?
- How do we partition the vector index?
- How do we cache embeddings?
- What happens if the LLM API fails?
- How do we handle duplicate requests?
- How do we make agent actions idempotent?
- How do we monitor hallucination?
- How do we roll back a prompt/model?

This turns your own engineering work into system-design practice.

12. YOUR INTERVIEW KNOWLEDGE MAP



classical	RAG	serving
deep learning	agents	monitoring
evaluation	eval	deployment

13. YOUR M.TECH SHOULD ALSO BE USED STRATEGICALLY

Your AI/ML master's is useful because the Applied AI/ML path explicitly values advanced quantitative/technical education. But the degree itself will not differentiate you. The differentiator is the combination of AI/ML education + engineering experience + production AI systems + cloud + DSA + system design.

14. YOUR WEEKLY SCOREBOARD

GOOGLE 12 MONTH TRACKER

Week | DSA | Project | ML | System Design | Interview

1 | 5 | 4 hrs | 3 | 1 | 1

2 | 6 | 5 hrs | 2 | 1 | 1

...

Every Sunday calculate:

- hours planned / hours completed
- DSA solved / DSA revisited
- project commits / tests added
- architecture docs / system designs
- mock interviews

Do not measure yourself by motivation. Measure output.

15. YOUR MILESTONE TARGETS

Checkpoint	What you should have
End of Month 3	120-150 DSA; FastAPI; PostgreSQL; testing; backend project; Python proficiency
End of Month 6	250 DSA; strong ML fundamentals; PyTorch; MLflow; Docker; RAG; evaluation; working Projects 1 and 2
End of Month 9	350 DSA; React; TypeScript; Kubernetes; GCP; CI/CD; agentic system; full-stack application
End of Month 12	~450 DSA; 3 major projects; GCP deployments; production observability; 10-20 system designs; 10+ mock interviews; strong STAR stories; Google-ready resume

16. WHEN SHOULD YOU START APPLYING?

Do not wait until Month 12. Start tracking Google openings around Month 6 and apply selectively during Months 7-12.

Month 6

|

start tracking

Month 7

|

first targeted applications

Month 8

|

resume refinement

Month 9

|

more applications + referrals

Month 10

|

interview heavy

Month 11

|

mock interview heavy

Month 12

|

full interview readiness

17. YOUR NETWORKING STRATEGY

Do not lead with “Hi, can you refer me?” Build relationships around technical work.

Possible posts:

- “I built a hybrid RAG system for engineering documentation and compared dense retrieval against BM25 + reranking. Here’s what happened...”
- “How I designed human-in-the-loop controls for an LLM agent.”
- “Designing canary deployment for ML models on Kubernetes.”
- “What I learned building an observable agent system.”

The profile should communicate: Software Engineer + AI/ML + Production systems + Technical thinking.

18. YOUR BIGGEST RISKS

Risk 1 - Tutorial addiction

You watch LangChain, RAG, Agents, Kubernetes and GCP content but do not build. Solution: 70% build, 20% study, 10% documentation.

Risk 2 - Too many projects

Do not build chatbots, face detectors, stock predictors, sentiment analyzers, image classifiers, PDF chatbots or movie recommenders as a portfolio pile. Build three deep projects.

Risk 3 - Ignoring DSA

AI projects will not save you if you cannot solve interview problems. DSA is non-negotiable.

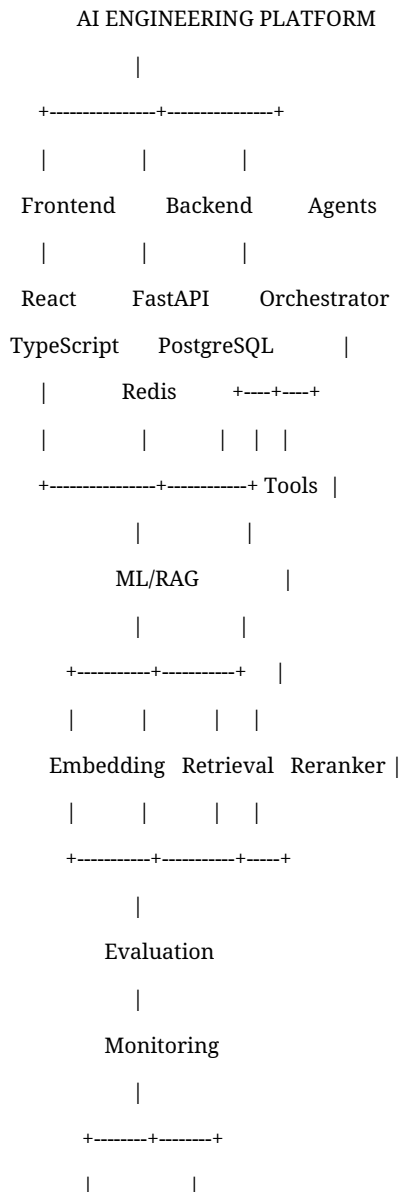
Risk 4 - Ignoring system design

At the experience level you are targeting, system design becomes increasingly important.

Risk 5 - Fake “production”

localhost + Streamlit + LLM API is not a production AI system. Build APIs, databases, queues, auth, tests, Docker, CI/CD, cloud deployment, logging, monitoring and failure handling.

19. THE SINGLE MOST IMPORTANT PROJECT ARCHITECTURE FOR YOU



GCP K8s CI/CD

That ecosystem gives you interview material for backend, frontend, databases, distributed systems, ML, GenAI, agents, MLOps, cloud, Kubernetes, testing, observability and architecture.

20. THE FINAL PROFILE I WANT YOU TO REACH

At the end of the year, a recruiter looking at your profile should immediately understand:

“This person is a software engineer first, who has specialized deeply into AI/ML and GenAI.”

Not: “AI enthusiast who knows Python.” And not: “Automation engineer who completed an AI degree.”

Software Engineering

|

Strong

v

Distributed / System Design

|

Strong

v

ML Engineering

|

Strong

v

GenAI / Agents

|

Strong

v

Cloud / MLOps

|

Strong

v

Domain expertise

Automotive / Engineering automation

Do not hide your automotive domain. Turn it into a differentiator. A coherent story can be: “I started close to embedded/ECU software and engineering automation. I then moved deeper into software automation, ML and AI systems. I built production-style systems that apply AI to engineering workflows.”

This can become a niche around AI for engineering systems across automotive, industrial, robotics, developer tools and engineering automation.

21. YOUR EXACT PRIORITY ORDER FOR THE NEXT 12 MONTHS

1. Python/software engineering
2. DSA
3. Backend
4. ML fundamentals

5. ML engineering
6. RAG
7. Agentic systems
8. React + TypeScript
9. GCP
10. Kubernetes
11. Observability
12. System design
13. Interview preparation
14. Resume/networking/applications

Governing principle: Every new technology must become part of a working system.

Learn Redis -> put Redis into the platform.

Learn Kubernetes -> deploy the platform.

Learn RAG -> put it into the platform.

Learn agents -> put agents into it.

Learn observability -> instrument it.

Learn system design -> redesign it for 10x scale.

Learn ML monitoring -> monitor its models.

That way, after 12 months you do not have 40 disconnected skills. You have one coherent engineering portfolio that demonstrates all of them.

22. YOUR FIRST 30 DAYS

Week	Focus
1	Python engineering; Arrays/hashmaps; FastAPI basics; Git repository setup
2	OOP/testing; Two pointers/sliding window; PostgreSQL; Document service
3	Async/concurrency; Binary search/stacks/queues; API testing; Document ingestion
4	Trees/recursion; logging; Docker; Complete Document Service v1

Then Month 2 begins the serious DSA + backend phase.

FINAL CHECKLIST

- One flagship AI engineering system built deeply and publicly
- One production ML platform with registry, serving, canary and drift
- One full-stack control-plane application
- ~450 carefully selected DSA problems with revision
- Strong Python + backend engineering
- Working React/TypeScript skills
- GCP + Kubernetes + CI/CD
- System-design practice using your own projects
- Production observability and reliability patterns
- Resume rewritten around software + AI/ML + production + architecture + impact

- GitHub portfolio with architecture, experiments and engineering documents
- Mock interview routine established by Month 12

TARGET ROLE LINKS

<https://www.google.com/about/careers/applications/jobs/results/100732318863237830-senior-software-engineer-aiml-genai-google-ads>

<https://www.google.com/about/careers/applications/jobs/results/136546867881091782-senior-software-engineer-aiml-google-cloud>

<https://www.google.com/about/careers/applications/jobs/results/101212497425375942-software-engineer-iii-full-stack-google-cloud>

<https://www.google.com/about/careers/applications/jobs/results/120884820812145350-senior-software-engineer-youtube-creator-support>

<https://www.google.com/about/careers/applications/jobs/results/91512172982477510-applied-aiml-engineer>

Note: role requirements can change. Re-check the live Google postings before each application cycle.